



Hardik Bajaj <hardikbajaj364@gmail.com>

Technical Screening Assessment – Full Stack Developer Intern

1 message

Proteccio Data Careers <careers@protecciodata.com>
Bcc: hardikbajaj364@gmail.com

2 July 2026 at 19:35

Dear Candidate,

Greetings!

Thank you for your interest in the **Full Stack Developer Intern** position. As part of our recruitment process, we would like to invite you to complete a practical technical assessment.

This assessment is designed to evaluate your ability to build a complete end-to-end application while demonstrating your understanding of software engineering best practices. We are not only interested in the final product but also in your approach to problem-solving, code quality, architecture, documentation, and development workflow.

Assessment Objective

The objective of this assignment is to assess your proficiency in:

- Frontend Development
- Backend Development
- Database Design and Schema Modeling
- REST API Design and Development
- Authentication and Authorization
- Validation and Error Handling
- Software Architecture
- Documentation
- Git Workflow and Version Control
- Overall Engineering Practices

This is intended to simulate a real-world development task that a Full Stack Developer may encounter in a professional environment.

Assessment Details

Position: Full Stack Developer Intern

Assessment Type: Practical Coding Assignment

Difficulty Level: Intermediate

Expected Completion Time: by EOD tomorrow (03 July 2026).

Submission Deadline: Please submit your completed assignment within the agreed timeline.

Business Scenario

Employee Leave Management System

A rapidly growing organization currently manages employee leave requests manually through emails and spreadsheets. This process has resulted in delays, inconsistent approvals, duplicate records, and limited visibility into employee leave status.

The organization has decided to digitize this process by developing a centralized Leave Management System.

As a Full Stack Developer Intern, your responsibility is to design and develop a Minimum Viable Product (MVP) that enables employees to submit leave requests while allowing managers to review, approve, or reject those requests efficiently.

Your implementation should emphasize scalability, maintainability, clean architecture, and a user-friendly interface.

Functional Requirements

Authentication

Develop a secure authentication mechanism with the following capabilities:

- Login using Email and Password
 - Session or JWT-based Authentication
 - Role-Based Access (Employee and Manager)
 - Protected Routes
 - Logout Functionality
 - Proper Validation
 - Error Handling for Invalid Credentials
-

Employee Module

Employees should be able to:

- Login securely
 - View Dashboard
 - Apply for Leave
 - View Leave History
 - Edit Pending Leave Requests
 - Cancel Pending Leave Requests
 - View Leave Status
 - Search Leave History
 - Filter Leave Records by Type or Status
-

Manager Module

Managers should be able to:

- Login securely
 - View Dashboard
 - View Pending Leave Requests
 - Review Leave Details
 - Approve Leave Requests
 - Reject Leave Requests with Comments
 - Search Employees
 - Filter Requests
 - View Employee Leave History
-

Dashboard

Employee Dashboard should include:

- Total Leave Requests
- Approved Requests
- Pending Requests
- Rejected Requests
- Recent Leave Activities

Manager Dashboard should include:

- Total Employees
 - Pending Approvals
 - Approved Requests
 - Rejected Requests
 - Recent Activities
-

Frontend Requirements

Develop a responsive and user-friendly interface.

Suggested Pages:

- Login
- Employee Dashboard
- Manager Dashboard
- Apply Leave
- Leave History
- Leave Details
- Pending Approvals

- Employee Profile
- Error (404) Page

Your implementation will be evaluated on:

- Responsive Design
 - UI/UX
 - Reusable Components
 - State Management
 - Client-side Validation
 - Error Handling
 - Loading Indicators
 - Code Organization
 - Accessibility Basics
-

Backend Requirements

Develop RESTful APIs with appropriate HTTP methods.

Suggested APIs include:

Authentication

- POST /login
- POST /logout

Employees

- GET /employees
- GET /employees/{id}

Leave Management

- POST /leaves
- GET /leaves
- GET /leaves/{id}
- PUT /leaves/{id}
- DELETE /leaves/{id}

Manager Operations

- GET /pending-leaves
- PUT /leaves/{id}/approve
- PUT /leaves/{id}/reject

Backend evaluation will focus on:

- API Design

- Modular Architecture
 - Authentication & Authorization
 - Input Validation
 - Exception Handling
 - Appropriate HTTP Status Codes
 - Logging
 - Security Best Practices
 - Code Maintainability
-

Database Design

Design a normalized database schema.

Suggested entities include:

Employee

- EmployeeID
- Name
- Email
- Password
- Department
- Role
- CreatedAt
- UpdatedAt

Leave

- LeaveID
- EmployeeID
- LeaveType
- StartDate
- EndDate
- Reason
- Status
- ManagerComments
- CreatedAt
- UpdatedAt

You will be evaluated on:

- Schema Design
- Normalization

- Relationships
 - Primary Keys
 - Foreign Keys
 - Constraints
 - Indexing Strategy
 - Scalability Considerations
-

API Documentation

Provide comprehensive API documentation including:

- Endpoint
- HTTP Method
- Description
- Request Parameters
- Request Body
- Sample Responses
- Status Codes
- Error Responses
- Authentication Requirements

Preferred options:

- Swagger/OpenAPI
 - Postman Collection
-

README Documentation

Your repository must include a well-written README containing:

- Project Overview
- Features
- Technology Stack
- Folder Structure
- Installation Steps
- Environment Variables
- Database Setup
- Backend Setup
- Frontend Setup
- Running the Application

- API Documentation
- Sample Login Credentials (if applicable)
- Assumptions
- Known Limitations
- Future Enhancements

The README should allow another developer to set up and run your application without requiring additional guidance.

Git Workflow

Please use Git throughout the assignment.

We expect to see:

- Meaningful Commit Messages
- Logical Commit History
- Feature-based Development
- Well-Organized Repository
- Proper Branching Strategy (optional but preferred)

Examples of good commits:

- Initial project setup
 - Configure database connection
 - Implement authentication module
 - Develop leave management APIs
 - Build responsive dashboard
 - Add frontend validations
 - Implement approval workflow
 - Update API documentation
 - Improve README
-

Project Structure

Your repository should be organized in a professional manner. A suggested structure is shown below:

- frontend/
- backend/
- database/
- docs/
- postman/
- README.md

- .env.example
 - .gitignore
-

Coding Standards

Your solution should demonstrate:

- Clean and readable code
 - Consistent naming conventions
 - Modular architecture
 - Reusable components
 - Proper separation of concerns
 - Secure coding practices
 - Input validation
 - Error handling
 - Configuration management
 - Maintainable project structure
-

Bonus Features

You may earn additional credit by implementing any of the following:

- JWT Refresh Tokens
 - Role-Based Access Control (RBAC)
 - Pagination
 - Search and Filtering
 - Docker Support
 - Unit Testing
 - Integration Testing
 - GitHub Actions (CI/CD)
 - Email Notifications
 - Leave Balance Calculation
 - Audit Logs
 - Mobile Responsive Design
 - Dark Mode
 - API Rate Limiting
-

Evaluation Criteria

Area	Marks
Frontend Development	20
Backend Development	25
Database Design	15
API Design & Documentation	10
README Documentation	10
Git Workflow & Code Quality	10
Project Structure	5
Bonus Features	5
Total	100

Minimum Passing Score: 70%

Submission Checklist

Please submit the following:

- GitHub Repository
 - Complete Source Code
 - README.md
 - Database Schema
 - API Documentation (Swagger or Postman Collection)
 - .env.example
 - Git Commit History
 - (Optional) 3–5 Minute Demo Video
-

Final Interview Discussion

Candidates shortlisted after the technical assessment will be invited for a technical discussion. During this session, you will be expected to explain:

- Your overall application architecture
- Technology stack selection
- Database schema and relationships
- API design decisions
- Authentication approach
- Challenges encountered during development
- Potential improvements if additional time were available

We encourage you to focus on writing clean, maintainable, and well-documented code. While leveraging publicly available resources for learning is acceptable, you should ensure that you fully understand and can explain every aspect of your implementation.

We wish you the very best with your assessment and look forward to reviewing your submission.

Best Regards,

Careers Team – [Proteccio Data](#)



Please do not reply to this email unless specifically requested. This mailbox may be used for recruitment and career communications only.

Privacy Notice: We use your name and email address to communicate and send updates. You can request changes or deletion of your information anytime. For more details, please visit the [Privacy Notice](#).

Disclaimer: This email may contain privileged and confidential information intended for the named recipient only. Unauthorized use or distribution is prohibited. Personal data is processed in compliance with applicable privacy laws. While we take reasonable security measures, we do not guarantee this email is free from viruses or unauthorized changes.

Privacy Notice: We use your name and email address to communicate and send updates. You can request changes or deletion of your information anytime. For more details please visit [Privacy Notice](#)

Disclaimer: This email may contain privileged and confidential information intended for the named recipient only. Unauthorized use or distribution is prohibited. Personal data is processed in compliance with applicable privacy laws. While we take reasonable security measures, we do not guarantee this email is free from viruses or unauthorized changes.